Arithmetic Operations on Indexed Concatenation Objects

Doing exact arithmetic on compressed numbers

Kenneth Caviness[1][a], Charles Sarr[2], Abigail Touma, Heaven Robles[4][b]

[1]*School of Engineering and Physics, Southern Adventist University, Taylor Circle, Collegedale, USA*
[2] *Laurelbrook Academy, Campus Drive, Dayton, Tennessee, USA*
[4]*Biology and Allied Health Department, Southern Adventist University, Taylor Circle, Collegedale, USA*
{caviness,heavenrobles}@southern.edu, charles.sarr@gmail.com

Keywords: Compressed Numbers, Concatenation, Indexed Concatenation, Lossless Compression, Operations.

Abstract: Indexed concatenation notation allows a lossless compact representation of integers and decimals, as well as of the strings, lists and matrices for which it was originally developed, requiring only some repetitive or describable structure. This paper shows that in many cases basic arithmetic operations can be performed on these compressed form objects without needing to unpack them first.

1 INTRODUCTION

The Indexed Concatenation (I-Cat) Notation (ICN) is more than merely a new way to represent data having inherent patterns in a compressed, reduced form. Some operations can be performed on these objects without completely "unpacking" them, producing an answer in I-Cat form. Examples of this include matrix addition and subtraction, and multiplication of a matrix by a constant. But we first consider integers and decimals written in I-Cat form, and what simple mathematical operations may be applied to them.

The originally intended usage for the ICN was to summarize and reduce (possibly nested) sequences of integers, but since concatenation is fundamental in the theory of computation and string concatenation, at least, is ubiquitous in computer languages, an obvious next step was the extension of the new notation from sequences to strings, and then applying it to "strings of numeric digits", i.e., to numbers, followed immediately. The introductory ICN paper included examples of integers and decimals in I-Cat form.

We introduce a greater variety of such examples of integers and both finite and infinite length decimals, with strictly repeating patterns or with more complicated, nested structure. The ICN lends itself admirably to summarizing these patterns, and it is easy to see how simple operations can be implemented on them. Such examples encourage us to extend the treatment to more challenging cases as well.

2 INTEGERS AND DECIMALS

Since any integer can be expressed as a string of digits, if a describable pattern exists in that string, it can be written in a compressed form. (For the present we limit our treatment to base-10, but usage with other bases is perfectly feasible.) We follow the design choices decided upon for the binary and indexed concatenation operators, as illustrated in equations (1) – (3).

$$\{1,2,3\} \# \{5,4,3\} \# \{7\} = \{1,2,3,5,4,3,7\} \quad (1)$$

$$\overset{3}{€}\text{"AB"} = \text{"AB"} \# \text{"AB"} \# \text{"AB"} = \text{"ABABAB"} \quad (2)$$

$$\overset{4}{\underset{i=1}{€}} S_i = S_1 \# S_2 \# S_3 \# S_4 \quad (3)$$

[a] https://orcid.org/0009-0008-5240-6260
[b] https://orcid.org/0009-0008-5434-7444

Table 1 shows several examples of integers and decimals, with their I-Cat forms. The examples highlight various useful features of the notation.

Table 1: Using the IC notation with numeric digits.

$1/3$	$=$	$0.333\ldots$	$= 0.\bar{3}$	$= 0.\overset{\infty}{€}3$
$2/9$	$=$	$0.222\ldots$	$= 0.\bar{2}$	$= 0.\overset{\infty}{€}2$
$1/7$	$=$	$0.142857142857\ldots$	$= 0.\overline{142857}$	$= 0.\overset{\infty}{€}(142857)$
$\frac{1706}{333}$	$=$	$5.123123123\ldots$	$= 5.\overline{123}$	$= 5.\overset{\infty}{€}(123)$
		1111111		$= \overset{7}{€}1$
		$10^6 = 1{,}000{,}000$		$= 1\overset{6}{€}0$
		$2 \times 10^9 = 2{,}000{,}000{,}000$		$= 2\overset{9}{€}0$
		$111{,}222{,}333{,}444$		$= \underset{i=1}{\overset{4}{€}}\left(\overset{3}{€}i\right)$
		1234567891011		$= \underset{k=1}{\overset{11}{€}}k$
		$0.010010001\ldots$		$= 0.\underset{i=1}{\overset{\infty}{€}}\left(\left(\underset{j=1}{\overset{i}{€}}0\right)1\right)$

For example, if an index is not needed to specify the digits or sequence of digits to be concatenated, it may be omitted. If no starting value for the index is given, it can be taken as 1 or 0, by agreed-upon convention. The upper limit for the concatenation index can be finite or infinity, as in the case of an infinitely repeating decimal. The concatenation is interpreted as applying to the digit (or string, list, etc.) immediately following it, otherwise parentheses are used. Finally, the ICN allows even non-repeating non-terminating decimals to be summarized concisely, provided a pattern can be identified. We are therefore not limited to the rationals, the terminating or repeating decimals that can be expressed as fractions.

3 SIMPLE OPERATIONS

It can easily be seen from Table 1 that simple additions/subtractions can be applied to the I-Cat objects, provided there is no borrow or carry extending beyond the repeating segment. For example, equation (4) shows an addition of I-Cats turned into a single I-Cat containing an addition.

$$0.\overset{\infty}{€}3 + 0.\overset{\infty}{€}2 = 0.\overset{\infty}{€}(3+2) = 0.\overset{\infty}{€}5 \tag{4}$$

This works exactly as it might reasonably be expected to:

$$\tfrac{1}{3} + \tfrac{2}{9} = \tfrac{3}{9} + \tfrac{2}{9} = \tfrac{5}{9} \tag{5}$$

Similarly, multiplying or dividing by a small integer may work smoothly, as in the case of $4(1/11) = 4/11$, given in equation (6).

$$4\left(0.\overset{\infty}{€}(09)\right) = 0.\overset{\infty}{€}(4 \times 09) = 0.\overset{\infty}{€}(36) \tag{6}$$

Both of these examples find their inspiration in the "obvious" addition and multiplication operations on exactly repeating decimals, such as:

$$0.333\ldots + 0.222\ldots = 0.555\ldots \tag{7}$$

$$4(0.090909\ldots) = 0.363636\ldots \tag{8}$$

In these examples there was no "carry", so the repeated segment of the I-Cat could be operated on without affecting the part of the number that preceded it – the "0.", in this case. It must also be noted that to add/subtract numbers with repeated segments of different length, say *n* and *m*, an additional step must be performed: rewriting each with a repeated segment of length LCM(*n*,*m*). Equations (9) and (10) illustrate this.

$$0.\overset{\infty}{€}(27) - 0.\overset{\infty}{€}(142857)$$
$$= 0.\overset{\infty}{€}(272727) - 0.\overset{\infty}{€}(142857) = 0.\overset{\infty}{€}(129870) \tag{9}$$

$$4.\overset{\infty}{€}(01) + 2.\overset{\infty}{€}(345) = 4.\overset{\infty}{€}(010101) + 2.\overset{\infty}{€}(345345)$$
$$= 6.\overset{\infty}{€}(355446) \tag{10}$$

For the critic who points out that the numbers in equations (9) and (10) could be simply written and operated on as fractions–yes, $3/11 - 1/7 = 10/77$ and $4\ 1/99 + 2\ 115/333 = 6\ 434/1221$ — we offer equation (11), involving irrational, yet clearly patterned numbers.

$$2\left(4.\underset{n=1}{\overset{\infty}{€}}\left(\left(\underset{k=1}{\overset{n!}{€}}1\right)3\right)\right) = 8.\underset{n=1}{\overset{\infty}{€}}\left(\left(\underset{k=1}{\overset{n!}{€}}2\right)6\right) \tag{11}$$

Unlike the irrational number included in Table 1, where the pattern of increasing number of 0s between the 1s could be recognized by writing 0.010010001 ..., here the upper limit of the nested I-Cat objects is a factorial, making the pattern difficult to identify by observation of any short initial segment. Yet the ICN allows the pattern to be summarized concisely – making it possible for us to use numbers the likes of which have never been written down before! (It

might, however, be justifiable to ask why we *want* to. Ahem! Moving right along….)

Commented [AT1]: Do you mean to ask or as asking?

Commented [KC2R1]: Oops. "to ask"

4 ADDING + COMPLICATIONS

Adding decimals in I-Cat form will frequently involve a "carry", where digits in either repeated or non-repeated parts affect other parts of the expression. But there are also alignment issues that may complicate matters. We start with some simple examples of the types of situations that must be considered.

4.1 ADDING NON-REPEATING SEGMENTS

If our numbers have non-repeating integer or decimal expansions, we need only align by the decimal point, pad with 0s any positions where needed, and add. Example:

$$0.5 + 0.125 = 0.500 + 0.125 = 0.625 \qquad (10)$$

Of course, this is simply ordinary decimal addition. The calculation for equivalent fractions gives the same result $\left(\frac{1}{2} + \frac{1}{8} = \frac{4}{8} + \frac{1}{8} = \frac{5}{8}\right)$, but uses a common denominator rather than zero-padding.

4.2 ADDING REPEATING SEGMENTS

The examples in Section 3 consisted of infinitely repeating segments, either of the same length or different lengths. So far, so good: either add corresponding parts, or partially expand one or both numbers to form matching segments of the same length. But we have not yet treated the situation where repeated segments do not align. For instance, consider

$$\begin{aligned} &222222220000 + 4444444 \\ &= (22222 \# 222 \# 0000) \\ &\quad + (00000 \# 444 \# 4444) \\ &= (22222 + 00000) \\ &\quad \# (222 + 444) \\ &\quad \# (0000 + 4444) \\ &= 22222 \# 666 \# 4444 \end{aligned} \qquad (11)$$

In ICN, this would be:

$$\begin{aligned} &\left(\overset{8}{€}2 \# \overset{4}{€}0\right) + \overset{7}{€}4 \\ &= \left(\overset{5}{€}2 \# \overset{3}{€}2 \# \overset{4}{€}0\right) + \left(\overset{5}{€}0 \# \overset{3}{€}4 \# \overset{4}{€}4\right) \\ &= \left(\overset{5}{€}2 \# \overset{3}{€}6 \# \overset{4}{€}4\right) \end{aligned} \qquad (12)$$

This may seem to be more unwieldy than the previous, expanded out version, but it can be extended to any length repeating (and non-repeating) segments.

Adding a non-repeating number to a repeating IC requires similar treatment. For example,

$$400 + \overset{9}{€}2 = \left(\overset{6}{€}0 \# 400\right) + \left(\overset{6}{€}2 \# 222\right)$$

$$= \left(\overset{6}{€}(0 + 2) \# (400 + 222)\right) = \overset{6}{€}2 \# 622 \quad (13)$$

Note that, in order to align parts and perform the addition, we must break the 9 repeated 2s into a length-3 non-repeating part, leaving only 6 copies in the explicitly repeating segment. In fact, 400 itself can be written with a repeating segment:

$$\left(4 \# \overset{2}{€}0\right) + \overset{9}{€}2$$

$$= \left(\overset{6}{€}0 \# 4 \# \overset{2}{€}0\right) + \left(\overset{6}{€}2 \# 2 \# \overset{2}{€}2\right)$$

$$= \overset{6}{€}2 \# 6 \# \overset{2}{€}2 \qquad (14)$$

Again, this might seem to be an unnecessarily difficult way to perform a simple operation, but it can be extended to any length repeating (and non-repeating) segments:

$$\left(4 \# \overset{20}{€}0\right) + \overset{99}{€}2$$

$$= \left(\overset{78}{€}0 \# 4 \# \overset{20}{€}0\right) + \left(\overset{78}{€}2 \# 2 \# \overset{20}{€}2\right)$$

$$= \left(\overset{78}{€}(0 + 2) \# (4 + 2) \# \overset{20}{€}(0 + 2)\right)$$

$$= \overset{78}{€}2 \# 6 \# \overset{20}{€}2 \qquad (15)$$

The process is exactly the same, but it can be applied to otherwise untreatably large segment lengths.

This process of breaking up one or both of the addends into segments with the same length (and number of repetitions, if repeated) allows addition despite alignment issues. We now turn to the treatment of "carries".

4.3 ADDING REPEATING AND NONREPEATING SEGMENTS

Consider an example with both repeating and nonrepeating parts. To complete the operation we need to make sure that the decimal point and the repeating and nonrepeating parts all line up.

$$\left(0.12 \# \overset{\infty}{€}7\right) + 0.\overset{\infty}{€}(011)$$

$$= \left(0.127\overset{\infty}{€}(777)\right) + \left(0.011\overset{\infty}{€}(011)\right)$$

$$= (0.127 + 0.011)\overset{\infty}{€}(777 + 011)$$

$$= (0.138)\overset{\infty}{€}(788)$$

4.4 ADDING WITH CARRIES

Once addends have been sized (broken up and/or padded), the last complication to consider is whether the addition of corresponding parts might result in a carry that spills over into another part. A simple example of this is $88888888 + 4000$. Again, we follow the strategy of formatting the addends in same-size blocks, but blindly following the strategy of adding up corresponding pieces would now cause a problem at the point marked by a not-equal sign:

$$\left(\overset{9}{€}8\right) + \left(4 \# \overset{3}{€}0\right) = \left(\overset{5}{€}8 \# 8 \# \overset{3}{€}8\right) + \left(\overset{5}{€}0 \# 4 \# \overset{3}{€}0\right)$$
$$\neq \left(\overset{5}{€}(8+0) \# (8+4) \# \overset{3}{€}(8+0)\right)$$
$$= \overset{5}{€}8 \# 12 \# \overset{3}{€}8 = 8888812888 \qquad (16)$$

Of course, the correct answer is 888892888. A partial unpacking of the I-Cat(s) affected by the carry can fix this:

$$= \left(\left(\overset{4}{€}8 \# 88\right) + \left(\overset{4}{€}0 \# 04\right)\right) \# \left(\overset{3}{€}8 + \overset{3}{€}0\right)$$
$$= \left(\overset{4}{€}(8+0) \# (88+04)\right) \# \overset{3}{€}8$$
$$= \overset{4}{€}8 \# 92 \# \overset{3}{€}8 \qquad (17)$$

How far must the affected I-Cat be unpacked? Since a carry at the units position can result in a carry at the tens position, etc., we might unpack it one position at a time, starting from the right, until no additional carry is produced (i.e., until the sum of corresponding parts of the addends has the same number of digits as the parts themselves), or the I-Cat is completely unpacked. The actual implementation could be accomplished by means of digit arrays for the addends and the sum. Any two-digit element in the sum array would be split into a carry (to be recursively added to higher-order digits) and a remaining digit. The following examples may clarify the process.

$$\left(\overset{9}{€}9\right) + (1) = \left(\overset{8}{€}9 \# 9\right) + \left(\overset{8}{€}0 \# 1\right)$$
$$= \left(\overset{7}{€}9 \# 99\right) + \left(\overset{7}{€}0 \# 01\right) = \cdots$$
$$= (999999999) + (000000001)$$
$$= 1000000000 = 1 \# \overset{9}{€}0 \qquad (18)$$

$$\left(\overset{4}{€}9 \# 8 \# \overset{4}{€}9\right) + (1) = \left(\overset{4}{€}9 \# 8 \# \overset{2}{€}9 \# 99\right) + (01)$$
$$= \cdots = \left(\overset{4}{€}9 \# 89999\right) + \left(\overset{4}{€}0 \# 00001\right)$$
$$= \left(\overset{4}{€}9 + \overset{4}{€}0\right) + (90000) = \overset{4}{€}9 \# 9 \# \overset{4}{€}0$$
$$= \overset{5}{€}9 \# \overset{4}{€}0 \qquad (19)$$

In eq. (18), the entire I-Cat had to be unpacked because of the continued propagation of the carry – worst case scenario. In eq. (19), unpacking terminates when no further carry occurs, and corresponding terms can be added. In both cases a less than ideal "unpack one and recheck" method was used, but perhaps that can be improved upon. Notice also the additional compaction step that occurs at the end of the process, an example of the more general case where successive I-Cats can be combined into one:

$$\overset{n}{€}(d_1 d_2 \ldots d_k) \# \overset{m}{€}(d_1 d_2 \ldots d_k) = \overset{n+m}{€}(d_1 d_2 \ldots d_k) \qquad (20)$$

The algorithm for adding numbers written in ICN, now consists of the following steps:

- aligning the numbers by the (actual or implied) decimal point,
- reformatting to equal lengths and number of repetitions of corresponding parts, and

Commented [AT3]: It may be helpful to expand this and actually show the carry throughout the operation.

Commented [KC4R3]: Yes, that sounds good. Try your hand at expanding it a bit, please.

- performing the addition by use of an array to process any carries, starting from the right and proceeding leftward.

Each element of this temporary "sum/carry array" must become single digit before it represents the actual sum. Table 1 begins redoing eq. (19) step-by-step, starting with addends already resized and with corresponding parts added in the first "Temp" line, and with implicit concatenation between the columns. Each manipulation of the temporary array is triggered by an element being longer than a single digit, the action that is then required is to replace the offending entry (here a "$^{1}0$") by its least significant digit (here "0") and adding the carry to the previous element. But

Addends:	$\overset{4}{€}9$	8	$\overset{3}{€}($	9	)	9
	$\overset{4}{€}0$	0	$\overset{3}{€}($	0	)	1
Temp:	$\overset{4}{€}9$	8	$\overset{3}{€}($	9	)	$^{1}0$
	$\overset{4}{€}9$	8	$\overset{3}{€}($	$^{1}0$	)	0
	$\overset{4}{€}9$	$8+1$	$\overset{3}{€}($	0	)	0
	$\overset{4}{€}9$	9	$\overset{3}{€}($	0	)	0

rather than unpacking the I-Cats one digit at a time, we notice that the carry into the rightmost of the I-Cat and the carry out of its leftmost digit are the same, allowing the shortcut shown in the following lines of Table 1. Here, a carry of "1" on the first Temp line generated the "digit" "$^{1}0$" inside the I-Cat, which is then replaced by "0" with a carry of "1". Since the incoming and outgoing carries are equal, we can proceed. Otherwise the I-Cat would need to be unpacked to treat its rightmost entry separately. (This won't occur under addition, but must be considered with multiplication – see below.)

5 MULTIPLICATIONS AGAIN

Multiplying a I-Cat numerical expression by an integer amounts to the same thing as repeated addition, which can easily result in carrying a digit greater than 1, or even multiple digits. We start by unpacking a certain number of iterations of each I-Cat in the expression, then follow basically the same plan for handling carries as was developed for addition. At the end, some repacking may be possible. Some cases change the number of non-repeated digits before the I-Cat, or in the case of finite expressions, after it. Again, a few examples are in order.

Consider multiplying 222233334444 by 4. Might the same procedure with a temporary "digit" array for the treatment of carries, as developed for addition of IC form numbers, also work here? Table 2 demonstrates the process. The multiplicand $\overset{4}{€}_{i=2}\left(\overset{4}{€}i\right)$ is first written without nested I-Cats and with one digit unpacked to check equality of carry in/out.

Multiplier:	4						
Multiplicand:	$\overset{4}{€}$	2		$\overset{4}{€}$	3	$\overset{4}{€}$	4
Temp:	$\overset{4}{€}$	8		$\overset{4}{€}$	$^{1}2$	$\overset{4}{€}$	$^{1}6$	^{0}
	$\overset{4}{€}$	8		$\overset{4}{€}$	$^{1}2$	$\overset{3}{€}$	$^{1}6$	$^{1}6$
	$\overset{4}{€}$	8		$\overset{4}{€}$	$^{1}2$	$\overset{3}{€}$	$^{1}7$	6
	$\overset{3}{€}$	8	8	$\overset{4}{€}$	$^{1}3$	$\overset{3}{€}$	7	6
	$\overset{3}{€}$	8	9	$\overset{4}{€}$	3	$\overset{3}{€}$	7	6
Answer:	$\overset{3}{€}$	8	9	$\overset{4}{€}$	3	$\overset{3}{€}$	7	6

The algorithm gives

$$4\left(\overset{4}{€}_{i=2}\left(\overset{4}{€}i\right)\right) = \overset{3}{€}8 \# 9 \# \overset{4}{€}3 \# \overset{3}{€}7 \# 6, or$$
$$4(222233334444) = 888933337776,$$

and that is the correct answer! It will be noticed that we unpacked the last digit of each IC form object in the multiplicand. That was because the last digit did not necessarily have an incoming carry applied to it, whereas all the others did. The algorithm can be formalized as follows:

*** Multiply the multiplier by the largest digit of the multiplicand (here, 4), subtract 1 from the length of the result (here the result was 16, its length 2, giving 1). This is the maximum number of digits affected by a single carry, and is the number of digits to unpack in each IC form in the multiplicand.

Alternatively, treat each string of digits inside an I-Cat or between I-Cats as one column, noting the number of digits that column can take.

Multiplication example with a carry greater than 1.

$9\left(\overset{9}{\underset{k=1}{€}}(765)\right)$:

Multiplier:		9								
Multiplicand (expanded)		$\overset{9}{€}$	(	7	6	5	)			
Temp:		$\overset{9}{€}$	(	$^{6}3$	$^{5}4$	$^{4}5$	)			
		$\overset{9}{€}$	(	$^{6}3$	$^{5}8$	5	)			
		$\overset{9}{€}$	(	$^{6}8$	8	5	)			
	6	$\overset{8}{€}$	(	8	8	5+6	)	8	8	5
	6	$\overset{8}{€}$	(	8	8	$^{1}1$	)	8	8	5
	6	$\overset{8}{€}$	(	8	9	1	)	8	8	5
Answer:	6	$\overset{8}{€}$	(	8	9	1	)	8	8	5

The algorithm gives

$$9\left(\overset{9}{\underset{k=1}{€}}\left(\overset{2}{\underset{i=0}{€}}(7-i)\right)\right) = 6\overset{8}{€}(891)885$$

An alternative is to allow the columns to have as many digits as the original numeric string has in each position, and only treat additional digits as carries:

Multiplier:		9				
Multiplicand (expanded)		$\overset{9}{€}$	(	765	)	
Temp:		$\overset{9}{€}$	(	$^{6}885$	)	
		$\overset{8}{€}$	(	$^{6}885$	)	$^{6}885$
Answer:	6	$\overset{8}{€}$	(	891	)	885

6 BORROWING

Slightly modifying the addition algorithm allows subtraction: "borrowing" is triggered when any "digit" of the temporary array is less than 0. The action taken is to add 10 to the current element and subtract 1 from the element to the left, iterating leftward, if necessary, also checking the equality of incoming/outgoing borrowing from an I-Cat as in the carrying shortcut. (If there is no element to the left to borrow from, the final answer is negative, and might better be treated using $a - b = -(b - a)$.) Tables 3 and 4 show examples of subtraction with borrowing: 3333333 – 4 and $20000 - 3$.

Subtrahends:	$\overset{6}{€}3$		3
	$\overset{6}{€}0$		4
Temp:	$\overset{6}{€}3$		-1
	$\overset{5}{€}3$	3	-1
	$\overset{5}{€}3$	3-1	-1+10
Answer:	$\overset{5}{€}3$	2	9

Subtrahends:	2	$\overset{3}{€}0$	0
	0	$\overset{3}{€}0$	3
Temp:	2	$\overset{3}{€}0$	-3
	2	$\overset{3}{€}(-1)$	-3+10
	2-1	$\overset{3}{€}(-1+10)$	7
Answer:	1	$\overset{3}{€}9$	7

Ex 9 x IC(103)

1.1.1 Charles' musings:

Because decimal alignment is going to be critical for addition/subtraction parsing is going to be a significant part of the process. We will need to break the decimal aligned addends into workable subsets that can then be reassembled. I see four cases: 1) nothing to constant or repeated section as in differing lengths of addends, NOOP; 2) constant to constant yields constant. Watch for carry*; 3) repeat to repeat, almost identical to #2; 4) constant to repeat which will yield either constant (most likely) or modified repeat under rare circumstances. Case 4 will depend on the placement of the constant within the repeat. It should be parched as constant against repeat for only the length of the repeat necessary to compute. The rest of the repeat should be handled as case 1. This gives us three sub cases. Constant over beginning, constant over ending, or worst case constant over mid repeat which breaks us into three segments. Then we have to reassemble all the pieces handling any carries/borrows and adjusting repeats as required by the carries and uniqueness of circumstances.

*carries are going to be the devil in the details. A carry into a constant is no problem but a carry into a repeat will break the repeat. Similar issue with borrowing

Constant + constant, constant + repeat => break up a repeat, repeat + repeat => if same length do it, unpack to LCM.

7 MULTIPYING DECIMALS

If two decimals, written in ICN, are multiplied, a situation arises that is comparable to multiplying two expressions given as indexed sums (finite or infinite). This is illustrated by equation ().

The reader is reminded of how such expressions are handled and simplified.

We hope to find a similar algorithm for multiplying I-Cats.

8 MATRIX I-CAT TECHNIQUES

This is reminiscent of matrix multiplication, written in indexed form.

9 MULTIPLYING DECIMALS – AND MATRICES? – IN ICN

9.1 Comparing multiplications

Both multiplication of two decimals in I-Cat form and the multiplication of two matrices involve an inner product operation, multiplying parts and summing them up. In addition for matrices in I-Cat form to be multiplied we need to finally notice the elephant in the room: our matrices are always grouped by *rows*.

9.2 Transpose of a matrix?

The elephant may be managed if we find a way to *transpose* a matrix written in I-Cat form.

10 CONCLUSIONS

11 PROSPECTS

Where do we go from here?

ACKNOWLEDGEMENTS

The authors acknowledge the generous support of the Academic Research Committee of Southern Adventist University for funding this project and related work which led to it.

REFERENCES

Our ICN Paper!
Our invited extended ICN Paper!

2 MOTIVATION

$$\left\{ \mathop{€}_{k=1}^{\infty} \left[\{1,1,2,2\}, \mathop{€}_{j=1}^{2} [\{2,2j\}],\ \{1,2k+3\}, \{1,1\}, \mathop{€}_{i=1}^{2k} [\{1,2k+8\}], \{\,\} \right] \right\} \quad (2)$$

Table 2: Network edges and the corresponding EDSL.

Edges:	{ 1→ 2, 1→ 2, 1→3, 1→ 3}	{2→ 4, 2→ 4}	{3→ 5, 3→ 7}	{4→5, 4→9}	{5→6, 5→6}	…
EDSL:	{1, 1, 2, 2}	{2, 2}	{2, 4}	{2, 4}	{1, 5}	…

3 NOTATION AND PRECEDENTS

We define the operators both for standard and indexed concatenation, explaining the background of the current indexed concatenation notation.

3.1 Binary (Infix) Operator

There is no generally accepted binary operator for concatenation.

3.2 Indexed Operator

Given a choice of precedents to follow, we opt for the following for indexed concatenation:

Table 3: Operation of infix concatenation on various datatypes.

$string$	:	"$a_1 a_2 \dots a_m$"	$\#$	"$b_1 b_2 \dots b_n$"	$=$	"$a_1 a_2 \dots a_m b_1 b_2 \dots b_n$"

$list$	:	$\{a_1, a_2, \dots, a_m\}$	$+\!\!+$	$\{b_1, b_2, \dots, b_n\}$	$=$	$\{a_1, a_2, \dots, a_m, b_1, b_2, \dots, b_n\}$
$sequence$	:	$[a_1, a_2, \dots, a_m]$	$+\!\!+$	$[b_1, b_2, \dots, b_n]$	$=$	$[a_1, a_2, \dots, a_m, b_1, b_2, \dots, b_n]$

.

5 SIMPLE EXAMPLES

We provide multiple examples of the capabilities of indexed concatenation when applied to lists, matrices, and other datatypes. In each case indexed concatenations shows promising potential to concisely summarize repeated patterns.

5.2 Integers or digits

The notation might even extend to concatenating the digits of numbers, when the argument is neither a list nor a sequence, but an integer: the concatenation of integers should be an integer.

$$\overset{5}{\underset{i=1}{€}} i^2 = 1 \# 4 \# 9 \# 16 \# 25 = 1491625. \qquad (15)$$

Table 4 provides several examples of compressing digits with indexed concatenation.

5.3 Lists and Sequences

We now turn to indexed concatenation of lists and sequenced, where the notation also works as expected.

$$\overset{3}{\underset{i=1}{€}} \{i, i^2\} = \{1,1\} \# \{2,4\} \# \{3,9\} = \{1,1,2,4,3,9\} \quad (17)$$

$$\overset{5}{\underset{i=1}{€}} \{i, 10-i\} = \{1,9\} \# \{2,8\} \# \{3,7\} \# \{4,6\} \# \{5,5\}$$
$$= \{1,9,2,8,3,7,4,6,5,5\} \qquad (18)$$

Again, the concatenation of lists is a list, without adding nested levels of the list structure.

$$\overset{3}{\underset{i=1}{€}} \{\{i, i^2\}\} = \{\{1,1\}\} \# \{\{2,4\}\} \# \{\{3,9\}\} = \{\{1,1\},\{2,4\},\{3,9\}\} \quad (19)$$

With sequences, the only difference is that all sequence delimiters that remain after the concatenation will vanish if inside another structure, so subsequences expand back out without adding additional levels of nesting. (Outer sequence delimiters remain, so the following is a sequence, not a list.)

$$\overset{3}{\underset{i=1}{€}} \overset{3}{\underset{j=1}{€}} [ji] = \overset{3}{\underset{i=1}{€}} [i, 2i, 3i] = [1,2,3] \# [2,4,6] \# [3,6,9]$$
$$= [1,2,3,2,4,6,3,6,9] \qquad (20)$$

But invoking the disappearance of our sequence-delimiter when inside some other object, one can concatenate subsequences in an outer list:

$$\left\{\overset{3}{\underset{i=1}{€}} \overset{3}{\underset{j=1}{€}} [ji]\right\} = \cdots = \{[1,2,3,2,4,6,3,6,9]\}$$
$$= \{1,2,3,2,4,6,3,6,9\} \qquad (21)$$

These can be nested to any level desired. So we have two ways to represent a list of integers having an internal pattern. For example,

$$\{1,1,1,1,1,2,2,2,2,1,1,1,1,1,2,2,2,2\}$$
$$= \left\{\overset{5}{€}[1], \overset{4}{€}[2], \overset{5}{€}[1], \overset{4}{€}[2], \overset{5}{€}[1], \overset{4}{€}[2]\right\}$$
$$= \left\{\overset{3}{€}\left[\overset{5}{€}[1], \overset{4}{€}[2]\right]\right\} \qquad (22)$$

$$\{1,1,1,1,1,2,2,2,2,1,1,1,1,1,2,2,2,2\}$$
$$= \{1,1,1,1,1\} \# \{2,2,2,2\} \# \{1,1,1,1,1\} \# \{2,2,2,2\}$$
$$= \overset{5}{€}\{1\} \# \overset{4}{€}\{2\} \# \overset{5}{€}\{1\} \# \overset{4}{€}\{2\}$$
$$= \overset{2}{€}\left(\overset{5}{€}\{1\} \# \overset{4}{€}\{2\}\right) \qquad (23)$$

$$\overset{10}{\underset{i=1}{€}} \{2^{10-i}\} = \{512,256,128,64,32,16,8,4,2,1\} \qquad (24)$$

$$\overset{3}{\underset{n=0}{€}} \left\{\overset{3-n}{\underset{i=0}{€}} \{3-i-n\}\right\} = \{\{3,2,1,0\},\{2,1,0\},\{1,0\},\{0\}\} \quad (25)$$

$$\{\{1\},\{1,1\},\{1,2,1\},\{1,3,3,1\},\{1,4,6,4,1\},\{1,5,10,10,5,1\}\} \quad (26)$$

5.4 Matrices

A generalized matrix (a_{ij}) with $1 \le i \le m$, $1 \le j \le n$, can be written in IC notation as a concatenated list of concatenated lists:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_n \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} = \overset{m}{\underset{i=1}{€}} \left\{\overset{n}{\underset{j=1}{€}} \{a_{ij}\}\right\} \qquad (27)$$

IC form:

If the matrix has some additional structure, that could also be shown explicitly in the IC form.

$$\begin{pmatrix} 0 & \sqrt{1} & 0 & 0 & 0 & 0 & 0 & \cdots \\ 0 & 0 & \sqrt{2} & 0 & 0 & 0 & 0 & \cdots \\ 0 & 0 & 0 & \sqrt{3} & 0 & 0 & 0 & \cdots \\ 0 & 0 & 0 & 0 & \sqrt{4} & 0 & 0 & \cdots \\ 0 & 0 & 0 & 0 & 0 & \sqrt{5} & 0 & \cdots \\ 0 & 0 & 0 & 0 & 0 & 0 & \sqrt{6} & \cdots \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix} = \overset{\infty}{\underset{k=1}{€}} \left\{\left\{\overset{k}{€}[0], \sqrt{k}, \overset{\infty}{€}[0]\right\}\right\}$$

Very concise. Similarly, the infinite matrix representations of the creation operator ($\hat{a}^\dagger$) the position ($\hat{x}$) and momentum ($\hat{p}$) operators, can all be written in nested IC form.

9 CONCLUSIONS

The Indexed Concatenation Notation, following the lead of the other indexed operators, allows compact representations of strings, lists, sequences, integer lists, matrices, and other similar expressions, examples of which we have considered above. It capitalizes on repetitions, periodicities and other patterns to compress information into a reduced form. It may be hoped that such notation summarizing internal patterns may not only serve as a new means of information compression, but also contribute to simpler and more insightful use of the data.

10 PROSPECTS

Where do we go from here?

1. Our code to partially or completely expand out Indexed Concatenation lists and sequences works for any finite nesting level, but:
   (a) the reverse direction is harder; our algorithm currently identifies and reduces 70% of the sample cases attempted.
   (b) treatment of strings and integers has not yet been implemented in either direction.

ACKNOWLEDGEMENTS

The authors acknowledge the generous support of the Academic Research Committee of Southern Adventist University for funding this project and related work which led to it.

REFERENCES

Our ICN Paper!

Campbell, J. (1971). Comparative survey of programming languages. *COMPUTING AS A LANGUAGE*, pages 391–484.

Cantor, G. (1874). Über eine Eigenschaft des Inbegriffes aller reellen algebraischen Zahlen. *Journal für diereine und angewandte Mathematik*, 77:258–262.

Caviness, K. (2011). Indexing strings and rulesets. *The Mathematica Journal*, 13.

de Bruijn, N. G. (1977). Notation for concatenation. Technical report, Technische Hogeschool Eindhoven.

Maheshwari, U. and Smid, M. (2024). *Introduction to Theory of Computation*. School of Computer Science, Carleton University.

Nachbar, R. (2023). Reply to changed methods for displaying graphs (networks). https://community.wolfram.com/groups/-/m/t/2862755?p_p_auth=iQ7ZaUTu.

Rehmann, U. (2020). Sequence. https://encyclopediaofmath.org/in-dex.php?title=Sequence&oldid=48671.

Sipser, M. (2012). *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition.

WsCubeTech (2025). C program to concatenate two strings (5 ways). https://www.wscubetech.com/resources/c-programming/programs/concatenate-two-strings.